Laboration 1

Group number: XXX
Vootele Mets
Juan Rodriguez
Marco Malosti
Björk Lucas
Sofia Fernandes
David Weingut

April 20, 2026

UPPSALA
UNIVERSITET

Contents

1 Problem statement **2**

2 Model and parameters **3**
2.1 Population equilibrium 4

3 Theoretical Background: Fixed Points, Periodic Points, and Newton's Method **6**
3.1 Fixed Points 6
3.2 Periodic Points 6
3.3 Reformulation as a Root-Finding Problem 7

4 Numerical method **8**
4.1 Bifurcation Cascade 8
4.2 Finding fixpoints in periodic orbits with Newtons Method 8

5 Results **10**
5.1 Bifurcation Cascade 10
5.2 p-Orbits 11
5.3 Algorithm 12
5.4 Comparison 14

6 Interpretation **15**

7 Conclusion **15**

8 Use of LLM tools **15**

1. Problem statement

State clearly what model or problem you study and what questions you want to answer. Added this changed version via upload. Changing the introduction.tex file.

2. Model and parameters

The model under investigation is

$$x_{n+1} = (1-\varepsilon)r_1 x_n(1-x_n) + \varepsilon r_2 y_n(1-y_n) \tag{2.1}$$
$$y_{n+1} = (1-\varepsilon)r_2 y_n(1-y_n) + \varepsilon r_1 x_n(1-x_n) \tag{2.2}$$
$$\tag{2.3}$$

which is equivalent to a non-linear perturbation of a linear map:

$$\mathbf{x}_{n+1} = \mathbf{F}(\mathbf{x}_n) = A\mathbf{x}_n + \mathbf{f}(\mathbf{x}_n) \tag{2.4}$$

with

$$A = \begin{bmatrix} (1-\varepsilon)r_1 & \varepsilon r_2 \\ \varepsilon r_1 & (1-\varepsilon)r_2 \end{bmatrix}, \quad \mathbf{f}(\mathbf{x}) = A\mathbf{g}(\mathbf{x}), \quad \mathbf{g}(\mathbf{x}) = -x^2\mathbf{e}_x - y^2\mathbf{e}_y \quad \mathbf{x} = \mathbf{x}_n \tag{2.5}$$

so

$$\mathbf{F}(\mathbf{x}) = A\mathbf{x} + \mathbf{f}(\mathbf{x}) = \begin{bmatrix} (1-\varepsilon)r_1 & \varepsilon r_2 \\ \varepsilon r_1 & (1-\varepsilon)r_2 \end{bmatrix} \begin{bmatrix} x \\ y \end{bmatrix} + \begin{bmatrix} (1-\varepsilon)r_1 & \varepsilon r_2 \\ \varepsilon r_1 & (1-\varepsilon)r_2 \end{bmatrix} \begin{bmatrix} -x^2 \\ -y^2 \end{bmatrix}. \tag{2.6}$$

Looking for fixpoints amounts to the root finding problem

$$\begin{aligned}
\mathbf{G}(\mathbf{x}) &= \mathbf{F}(\mathbf{x}) - \mathbf{x} && (2.7)\\
&= A\mathbf{x} + \mathbf{f}(\mathbf{x}) - \mathbf{x} && (2.8)\\
&= A\mathbf{x} + A\mathbf{g}(\mathbf{x}) - \mathbf{x} && (2.9)\\
&= A(\mathbf{x} + \mathbf{g}(\mathbf{x})) - \mathbf{x} && (2.10)\\
&= 0 && (2.11)\\
\Rightarrow \mathbf{x} + \mathbf{g}(\mathbf{x}) &= A^{-1}\mathbf{x} && (2.12)\\
\iff \mathbf{g}(\mathbf{x}) &= (A^{-1} - I)\mathbf{x} && (2.13)\\
A^{-1} &= \frac{1}{\det(A)}\mathrm{adj}(A) = \frac{1}{\det(A)} \begin{bmatrix} (1-\varepsilon)r_2 & -\varepsilon r_1 \\ -\varepsilon r_2 & (1-\varepsilon)r_1 \end{bmatrix}^T && (2.14)\\
\det(A) &= (1-\varepsilon)^2 r_1 r_2 - \varepsilon^2 r_1 r_2 = (1-2\varepsilon)r_1 r_2 && (2.15)\\
\iff \begin{bmatrix} -x^2 \\ -y^2 \end{bmatrix} &= \begin{bmatrix} a & b \\ c & d \end{bmatrix} \begin{bmatrix} x \\ y \end{bmatrix} && (2.16)
\end{aligned}$$

$$\begin{cases}
a = \frac{1-\varepsilon}{r_1(1-2\varepsilon)} - 1 = \frac{\varepsilon-1}{2r_1(\varepsilon-1/2)} - 1 \\
b = \frac{-\varepsilon}{r_1(1-2\varepsilon)} = \frac{\varepsilon}{2r_1(\varepsilon-1/2)} \\
c = \frac{-\varepsilon}{r_2(1-2\varepsilon)} = \frac{\varepsilon}{2r_2(\varepsilon-1/2)} \\
d = \frac{1-\varepsilon}{r_2(1-2\varepsilon)} - 1 = \frac{\varepsilon-1}{2r_2(\varepsilon-1/2)} - 1
\end{cases} \tag{2.17}$$

$$\begin{cases}
x^2 + ax + by = x(x+a) + by = 0 \\
y^2 + cx + dy = cx + y(y+d) = 0
\end{cases} \tag{2.18}$$

Thus the fixpoints belong to the intersections of 2 parabolic surfaces. The simple nullcline (that is if we ought to find a fixpoint it is necessary for it to lie on this curve) is given by the first equation above

$$y(x) = \frac{-x^2 - ax}{b} \tag{2.19}$$

Inserting Eq (2.19) in the lower equation in (2.18) gives the following quartic polynomial:

$$p(x) = x^4 + 2ax^3 + (a^2 - bd)x^2 + b(cd - ad)x \tag{2.20}$$

from which we can readily factor x and thus $\mathbf{x}_1 = (0, 0)$ is the trivial root, and leaving

$$q(x) = x^3 + 2ax^2 + (a^2 - bd)x + bd(c - a) \tag{2.21}$$

since $\deg(q) = 3$ and $y(x)$ one-to-one for $b \neq 0$ then we can at most have 4 fixed points. In our case we might have even less than 4 fix points since we also treat population rates on the square $[0, 1]^2$.

2.1. Population equilibrium

If we consider the line parametrisation $x = y = t$ where $t \in [0, 1]$, which is the linear subspace of population rates of equal magnitude, we could ask if there is a rate t^* such that $\mathbf{G}(t^*) = 0$? Let $x = y = t$, then

$$\begin{aligned}
\mathbf{G}(\mathbf{x}) &= A\mathbf{x} + A\mathbf{g}(\mathbf{x}) - \mathbf{x} = 0 && (2.22)\\
&= A\mathbf{x} + A\mathbf{g}(\mathbf{x}) - \mathbf{x} && (2.23)\\
&= \mathbf{x} + \mathbf{g}(\mathbf{x}) - A^{-1}\mathbf{x} && (2.24)\\
&= \mathbf{g}(\mathbf{x}) + \mathbf{x} - A^{-1}\mathbf{x} && (2.25)\\
&= -t^2 \begin{bmatrix} 1 \\ 1 \end{bmatrix} + t \begin{bmatrix} 1 \\ 1 \end{bmatrix} - \frac{1}{(1-2\varepsilon)r_1 r_2} \begin{bmatrix} (1-\varepsilon)r_2 & -\varepsilon r_2 \\ -\varepsilon r_1 & (1-\varepsilon)r_1 \end{bmatrix} \begin{bmatrix} 1 \\ 1 \end{bmatrix} && (2.26)\\
&= -t^2 \begin{bmatrix} 1 \\ 1 \end{bmatrix} + t \begin{bmatrix} 1 - \frac{1}{r_1} \\ 1 - \frac{1}{r_2} \end{bmatrix} = \begin{bmatrix} 0 \\ 0 \end{bmatrix}. && (2.27)
\end{aligned}$$

If we disregard the trival fixpoint $t \neq 0$, then we have that

$$\begin{cases} t = 1 - \frac{1}{r_1} \\ t = 1 - \frac{1}{r_2} \end{cases} \tag{2.28}$$

which can hold simultaneously only if $r_1 = r_2 = r$. Thus, except for the trivial fixpoint then we can have a fixpoint of equally large population rates if the growth factors of both populations are equal. The second fixpoint in the diagonal case is thus along a line through the origin $\mathbf{x} = \lambda(r)\mathbf{v}$:

$$\mathbf{x}_1(r) = \left(1 - \frac{1}{r}\right) \begin{bmatrix} 1 \\ 1 \end{bmatrix} \tag{2.29}$$

with the constraint $1 \leq r$ since $(x, y) \in [0, 1]^2$. The nontrivial diagonal fixed pointsa form a one-parameter family parametrized by $1 \leq r$, corresponding to a one-dimensional branch of equilibria. Now we could linearize $\mathbf{G}$ around $\mathbf{x}_1$ to see wether $\mathbf{x}_1$ is an isolated root. We do this by computing the Jacobian (for $x = y = t$ and $r_1 = r_2 = r$):

$$\frac{\partial \mathbf{G}}{\partial \mathbf{x}} = \frac{\partial \mathbf{F}}{\partial \mathbf{x}} - \frac{\partial \mathbf{x}}{\partial \mathbf{x}} \tag{2.30}$$

$$= \frac{\partial \mathbf{F}}{\partial \mathbf{x}} - I \tag{2.31}$$

$$= \begin{bmatrix} (1-\varepsilon)r(1-2t) & \varepsilon r(1-2t) \\ \varepsilon r(1-2t) & (1-\varepsilon)r(1-2t) \end{bmatrix} - I \tag{2.32}$$

$$= r(1-2t)\begin{bmatrix} (1-\varepsilon) & \varepsilon \\ \varepsilon & (1-\varepsilon) \end{bmatrix} - I \tag{2.33}$$

$$= (2-r)\begin{bmatrix} (1-\varepsilon) & \varepsilon \\ \varepsilon & (1-\varepsilon) \end{bmatrix} - I \tag{2.34}$$

$$\lambda^2 - \operatorname{tr}\left(\frac{\partial \mathbf{G}}{\partial \mathbf{x}}\right) + \det\left(\frac{\partial \mathbf{G}}{\partial \mathbf{x}}\right) = 0 \tag{2.35}$$

$$\lambda_\pm = \frac{1}{2}\operatorname{tr}\left(\frac{\partial \mathbf{G}}{\partial \mathbf{x}}\right) \pm \sqrt{\frac{1}{4}\operatorname{tr}\left(\frac{\partial \mathbf{G}}{\partial \mathbf{x}}\right)^2 - \det\left(\frac{\partial \mathbf{G}}{\partial \mathbf{x}}\right)} \tag{2.36}$$

$$= (2-r)(1-\varepsilon) - 1 \pm \varepsilon(2-r) \tag{2.37}$$

$$\lambda_+ = 1 - r \tag{2.38}$$

$$\lambda_- = (2-r)(1-2\varepsilon) - 1 \tag{2.39}$$

Since $\det\left(\frac{\partial \mathbf{G}}{\partial \mathbf{x}}\right) = \lambda_+\lambda_-$, we know by the inverse function theorem that $\mathbf{x}_1$ is an isolated zero to $\mathbf{G}$ as long as neither $\lambda_+ = 0$ or $\lambda_- = 0$, otherwise the linearization becomes inconclusive and another branch, possibly off-diagonal can bifurcate from $\mathbf{x}_1$. To se wether we have fixed points for the case $r_1 \neq r_2$ we can consider the uncoupled case $\varepsilon = 0$

$$\begin{bmatrix} r_1 & 0 \\ 0 & r_2 \end{bmatrix}\begin{bmatrix} x(1-x) \\ y(1-y) \end{bmatrix} - \begin{bmatrix} x \\ y \end{bmatrix} = \begin{bmatrix} 0 \\ 0 \end{bmatrix} \tag{2.40}$$

Then,

$$\mathbf{x}_2(r_1, r_2) = \left(1 - \frac{1}{r_1}, 1 - \frac{1}{r_2}\right) \tag{2.41}$$

which is off-diagonal and away from the singularity $\varepsilon = 1/2$ found in the determinant in Equation (2.14).

3. Theoretical Background: Fixed Points, Periodic Points, and Newton's Method

A discrete-time dynamical system describes the evolution of a state variable through successive iterations of a given map. In this project's model, the state of the system n is described by the vector

$$z_n = (x_n, y_n),$$

where x_n and y_n denote the normalized densities of two interacting populations. The evolution of the system is determined by a map F, such that

$$z_{n+1} = F(z_n).$$

3.1. Fixed Points

A fixed point of a map is a state that remains unchanged under one application of the map. Thus, a point (x^*, y^*) is a fixed point of F if

$$F(x^*, y^*) = (x^*, y^*).$$

In this context, fixed points correspond to equilibrium states, where the reproductive dynamics and the interaction between the two populations are balanced. This reflects in a constant population density from one generation to the next. So, by definition, in order to get the fixed points, one has to find the roots of the system.

3.2. Periodic Points

Periodic points generalize the concept of fixed points. A point (x, y) is said to be periodic with period p if it returns to itself after p applications of the map:

$$F^p(x, y) = (x, y),$$

For example,

$$F^2(x, y) = F(F(x, y)),$$

and

$$F^3(x, y) = F(F(F(x, y))).$$

A period-2 and 3 points therefore satisfy, respectively,

$$F^2(x, y) = (x, y)$$

and

$$F^3(x, y) = (x, y),$$

meaning that the orbit repeats after three iterations.

It is important, however, to distinguish between a point satisfying $F^p(x, y) = (x, y)$ and a point having *minimal period p*. The minimal period is the smallest positive integer k such that

$$F^k(x, y) = (x, y).$$

For example, every fixed point satisfies

$$F^2(x, y) = (x, y),$$

but it should not be classified as a genuine period-2 point, because it already returns to itself after one iteration. Therefore, after solving the equation $F^p(x, y) = (x, y)$, it is necessary to verify that the solution does not have a smaller period.

3.3. Reformulation as a Root-Finding Problem

The numerical computation of periodic points can be formulated as a nonlinear root-finding problem. This problem is solved with the aid of Newton's method and, for that, it is necessary to adapt the problem. For a given period p, the function is defined as

$$G_p(x, y) = F^p(x, y) - (x, y).$$

To determine id a point is periodic, the following condition has to be met:

$$G_p(x, y) = \begin{pmatrix} 0 \\ 0 \end{pmatrix}.$$

This way, the problem is reduced strictly to finding the zeros of the function without the occurrence of repeated orbits. For $p = 2$, it solves

$$G_2(x, y) = F^2(x, y) - (x, y) = 0,$$

while for $p = 3$, it solves

$$G_3(x, y) = F^3(x, y) - (x, y) = 0.$$

This reformulation is useful and necessary because the iterated maps F^2 and F^3 generate nonlinear systems that are generally too complicated to solve analytically. A numerical method is therefore required.

4. Numerical method

Describe the numerical procedure used in your code. Mention the main parameter values, the number of iterates, the discarded transient, and how the figures were produced.

4.1. Bifurcation Cascade

For implementation of the map we use the function `coupled_map` from `common.py`. This function calculates the next generation of x and y values from the given x and y values, r_1, r_2 and ϵ. As we need more than one time step, we created the function `calc_timeline`, which returns the x and y values for a certain r_2 value. This function also has the option to choose different numbers of steps and discard a certain number of timesteps from the beginning of the time series.

Additionally, we set our constants to the values given in the task: $r_1 = 3.1$ and $\varepsilon = 0.3$. For r_2 we created a linspace from 2.8 to 3.97 with 800 values. The x and y values each iteration starts from are set to $x_0 = 0.2$ and $y_0 = 0.6$. We decided to use 400 iterations and discard the first 150.

For each value in the r_2 linspace we then calculate the time series and save the x values. We then use `plt.plot` to plot all these values and then move to the next r_2 value. Lastly, we added labels and the title and saved the figure.

4.2. Finding fixpoints in periodic orbits with Newtons Method

Newton's Method

Newton's method comes to solve this problem. It is an iterative numerical procedure for solving nonlinear equations. In one dimension, it is used to solve equations of the form

$$g(x) = 0,$$

which deeply resonates with the problem at hand.

Starting from an initial approximation x_0, Newton's method constructs a sequence of approximations according to

$$x_{n+1} = x_n - \frac{g(x_n)}{g'(x_n)}.$$

The method is based on a local linear approximation of the function. At each iteration, the function g is approximated by its tangent line at the current point x_n, and the zero of this tangent line is used as the next approximation. When the initial guess is sufficiently close to a true root and the derivative is nonzero, Newton's method converges.

In the present problem, the unknown is a two-dimensional vector, where

$$z = \begin{pmatrix} x \\ y \end{pmatrix}.$$

The function to be solved is therefore:

$$G_p(x, y) = \begin{pmatrix} G_{p,1}(x, y) \\ G_{p,2}(x, y) \end{pmatrix}.$$

Newton's method can then be applied through a system of two nonlinear equations,

$$G_{p,1}(x, y) = 0,$$
$$G_{p,2}(x, y) = 0.$$

In this case, naturally, the derivative needs to be replaced by the Jacobian matrix of G_p:

$$J_p(x, y) = DG_p(x, y) = \begin{pmatrix} \frac{\partial G_{p,1}}{\partial x} & \frac{\partial G_{p,1}}{\partial y} \\ \frac{\partial G_{p,2}}{\partial x} & \frac{\partial G_{p,2}}{\partial y} \end{pmatrix}.$$

At each iteration, the Newton correction s_n is obtained by solving the linear system

$$J_p(x_n, y_n)s_n = -G_p(x_n, y_n).$$

The approximation is then updated according to

$$\begin{pmatrix} x_{n+1} \\ y_{n+1} \end{pmatrix} = \begin{pmatrix} x_n \\ y_n \end{pmatrix} + s_n.$$

In the implemented code, the Jacobian matrix is computed symbolically, and at each Newton iteration the corresponding linear system is solved numerically. This produces a correction vector that moves the current approximation closer to a root of G_p. In the present implementation, the damping parameter is set equal to 1, corresponding to the standard Newton method.

Newton's method is a local method, meaning that its convergence depends strongly on the choice of initial condition. Different initial guesses may converge to different roots, fail to converge, or lead to points outside the meaningful region. For this reason, the program applies Newton's method to a grid of initial points in the unit square.

After the Newton iteration terminates, the resulting point must be verified. This is necessary because a small Newton step does not always guarantee that the point is an actual root of G_p. The program therefore evaluates the residual

$$\|G_p(x, y)\|.$$

A point is accepted only if this residual is below a defined tolerance. This ensures that the computed point satisfies

$$F^p(x, y) \approx (x, y).$$

In addition, the program only accepts solutions within the meaningful domain, the unit square values,

$$0 \le x \le 1, \qquad 0 \le y \le 1.$$

Finally, the minimal period of the point is checked. This step is essential because the solutions may include points of smaller period. For example, fixed points are also solutions of the period-2 equation. Therefore, the code tests the smallest integer $k \le p$ for which

$$F^k(x, y) \approx (x, y).$$

Only points whose minimal period is exactly p are retained as actual period-p points.

5. Results

5.1. Bifurcation Cascade

Figure 1 shows the bifurcation cascade of the coupled logistic map. At the lowest r_2 values, from 2.8 to around 2.87, we can see one branch. From $r_2 \approx 2.85$ the branch starts splitting into two and these branches separate from each other. At $r_2 \approx 3.7$ both branches split again so that we have four different branches. Each branch splits again at $r_2 \approx 3.9$. At slightly higher r_2 values the branches start splitting faster so that we end up with a point cloud that grows larger with higher r_2 values.

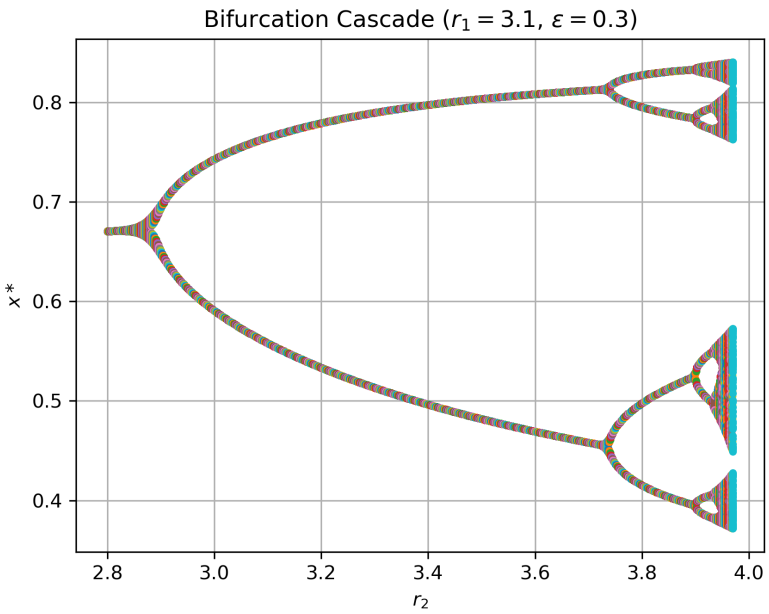
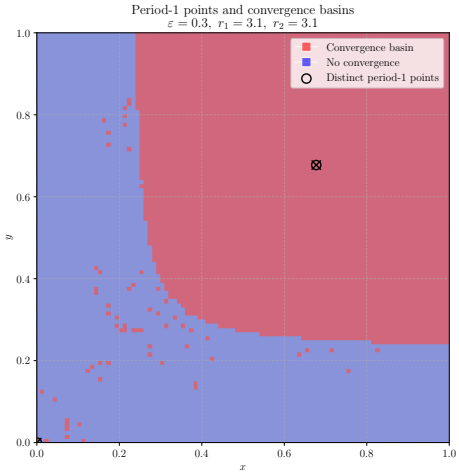
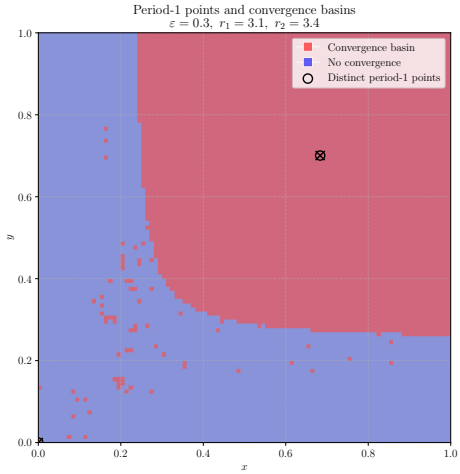


Figure 1. Bifurcation cascade for $r_1 = 3.1$ and $\epsilon = 0.3$

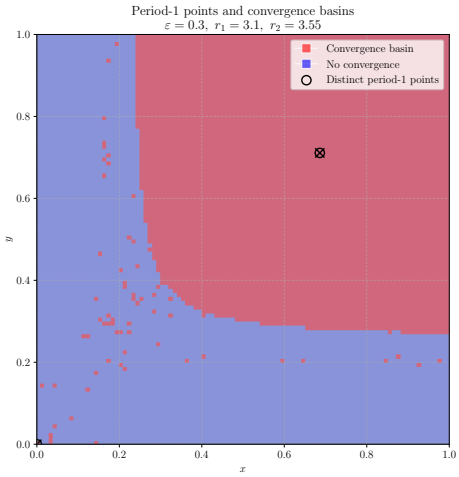
5.2. p-Orbits



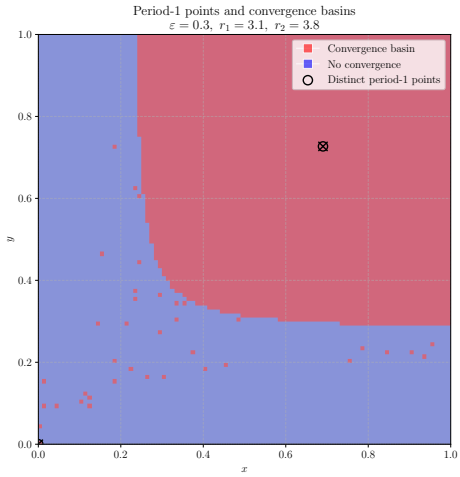
(a) `[0.67741935 0.67741935]`



(b) `[0.68345666 0.70051177]`

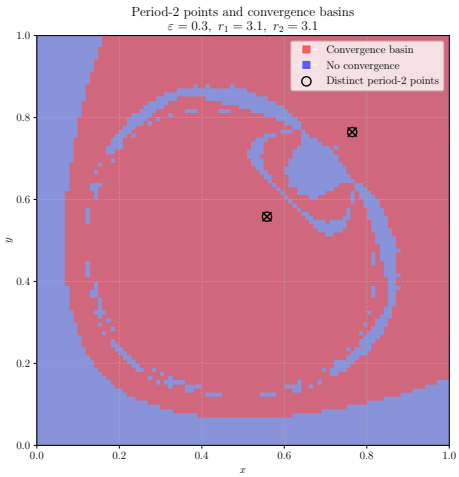


(c) `[0.68615866 0.71094441]`

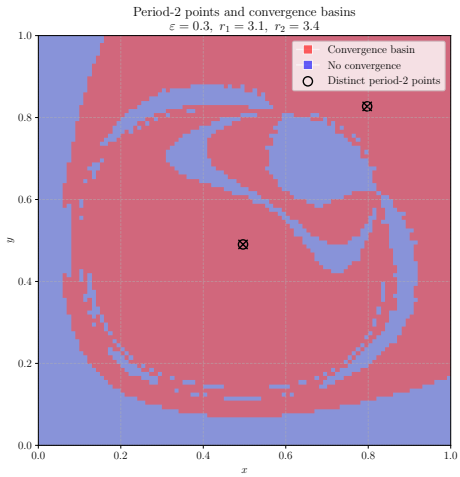


(d) `[0.69025989 0.72689488]`

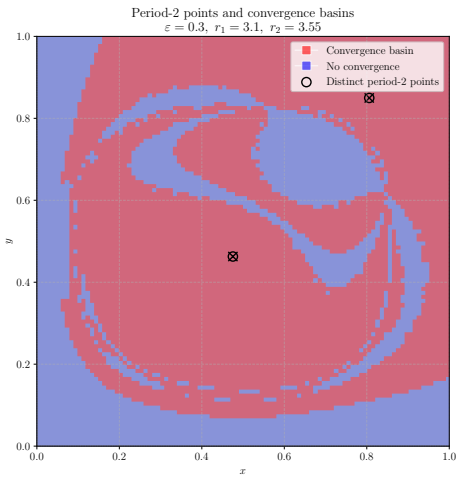
Figure 2. p-orbit behaviour for first parameter sweep



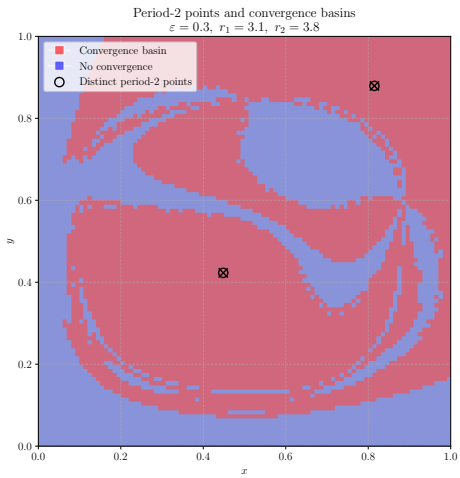
(a) `[0.76456652 0.76456652]`, `[0.55801413 0.55801413]`



(b) `[0.79737624 0.82726612]`, `[0.49635601 0.49035224]`



(c) `[0.80595371 0.84973562]`, `[0.47535588 0.46274208]`



(d) `[0.8149268 0.87917722]`, `[0.4483779 0.42282117]`

Figure 3. p-orbit behaviour for second parameter sweep

5.3. Algorithm

```
#include <mpi.h>
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>

int main(int argc, char *argv[]) {
    int rank, size;
    double a, b;
    MPI_Status status;

    MPI_Init(&argc, &argv); /* Initialize MPI                */

    MPI_Comm_size(MPI_COMM_WORLD, &size); /* Get the number of processors */
    MPI_Comm_rank(MPI_COMM_WORLD, &rank); /* Get my number                */


    a = (double) rank;
```

UPPSALA
UNIVERSITET

Figure 4. Replace this caption with a description of your result.

```
    if (size != 2) {
        /* This if-block makes sure only two processors
                            * takes part in the execution of the code, pay no
                            * attention to it */
        if (rank == 0)
            fprintf(stderr, "\aRun on two processors only!\n");

        MPI_Finalize();
        exit(0);
    }

    /* Here comes the important bit */

    if (rank == 0) {
        MPI_Ssend(&a, 1, MPI_DOUBLE, 1, 111, MPI_COMM_WORLD);
        MPI_Recv(&b, 1, MPI_DOUBLE, 1, 222, MPI_COMM_WORLD, &status);
        printf("Processor 0 got %f from processor 1\n", b);
    } else {

        MPI_Ssend(&a, 1, MPI_DOUBLE, 0, 222, MPI_COMM_WORLD);
        MPI_Recv(&b, 1, MPI_DOUBLE, 0, 111, MPI_COMM_WORLD, &status);
        printf("Processor 1 got %f from processor 0\n", b);
    }

    MPI_Finalize(); /* Shut down and clean up MPI */

    return 0;
}
```

5.4. Comparison

Figure 5 compares different sorting algorithm implementations.

```python
def bubble_sort(arr):
    """Bubble sort - O(n²)"""
    n = len(arr)
    for i in range(n):
        for j in range(0, n - i - 1):
            if arr[j] > arr[j + 1]:
                arr[j], arr[j + 1] = arr[j + 1], arr[j]
    return arr

# Example
numbers = [64, 34, 25, 12, 22, 11, 90]
print("Bubble sort:", bubble_sort(numbers.copy()))
```

(a) Bubble sort algorithm

```python
def merge_sort(arr):
    """Merge sort - O(n log n)"""
    if len(arr) <= 1:
        return arr

    mid = len(arr) // 2
    left = merge_sort(arr[:mid])
    right = merge_sort(arr[mid:])

    return merge(left, right)

def merge(left, right):
    result = []
    i = j = 0

    while i < len(left) and j < len(right):
        if left[i] <= right[j]:
            result.append(left[i])
            i += 1
        else:
            result.append(right[j])
            j += 1

    result.extend(left[i:])
    result.extend(right[j:])
    return result

# Example
numbers = [64, 34, 25, 12, 22, 11, 90]
print("Merge sort:", merge_sort(numbers.copy()))
```

(b) Merge sort algorithm

Figure 5. Comparison of sorting algorithms with different time complexities

6. Interpretation

Explain what the numerical results show. Connect the figures with the mathematical or modeling questions from the project.

7. Conclusion

Summarize the main conclusions in a few sentences.

8. Use of LLM tools

If you used an LLM in a meaningful way, briefly state:

- which prompts were important,
- what output was useful,
- what had to be corrected or verified.

If you did not use an LLM, state that explicitly.

Files submitted

List the files included in the submission.

Test. According to Source 1[2], and Source 2[1], and Source 3[2]. Test 3. Test 4. Test 5. Test 6.

References

[1] Alexandre Ern and Jean-Luc Guermond. *Finite Elements III*. Springer, 2021.

[2] Hans Petter Lantangen and Anders Logg. *Solving PDEs in Python – The FEniCS Tutorial I*. Springer Open, 2016.